Week-7

1.PROBLEM STATEMENT:

Design a parking system for a parking lot. The parking lot has three kinds of parking spaces: big, medium, and small, with a fixed number of slots for each size.

IMPLEMENT THE PARKINGSYSTEM CLASS:

- ParkingSystem(int big, int medium, int small) Initializes object of the ParkingSystem class. The number of slots for each parking space are given as part of the constructor.
- bool addCar(int carType) Checks whether there is a parking space of carType for the car that wants to get into the parking lot. carType can be of three kinds: big, medium, or small, which are represented by 1, 2, and 3 respectively. A car can only park in a parking space of its carType. If there is no space available, return false, else park the car in that size space and return true.

Example 1:

Input

["ParkingSystem", "addCar", "addCar", "addCar", "addCar"]

[[1, 1, 0], [1], [2], [3], [1]]

Output

[null, true, true, false, false]

Constraints:

- 0 <= big, medium, small <= 1000
- carType is 1, 2, or 3
- At most 1000 calls will be made to addCar

Source code:

```
class ParkingSystem {

    int big;
    int medium;
    int small;

    // Constructor
```

```
public ParkingSystem(int big, int medium, int small) {
    this.big = big;
    this.medium = medium;
    this.small = small;
}

public boolean addCar(int carType) {

    if (carType == 1) {
        if (big > 0) {
            big--;
            return true;
        }
    }

    if (carType == 2) {
        if (medium > 0) {
            medium--;
            return true;
        }
    }

    if (carType == 3) {
        if (small > 0) {
            small--;
            return true;
        }
    }
```

```
        return false;
    }
}
```

Input

["ParkingSystem","addCar","addCar","addCar","addCar"]

[[1,1,0],[1],[2],[3],[1]]

Output

[null,true,true,false,false]

2.PROBLEM STATEMENT:

The Java *instanceof* operator is used to test if the object or instance is an instanceof the specified type.

In this problem we have given you three classes in the editor:

- Student class
- Rockstar class
- Hacker class

In the main method, we populated an *ArrayList* with several instances of these classes. *count* method calculates how many instances of each type is present in the ArrayList. The code prints three integers, the number of instance of Student class, the number of instance of Rockstar class, the number of instance of Hacker class.

But some lines of the code are missing, and you have to fix it by modifying only lines! Don't add, delete or modify any extra line.

To restore the original code in the editor, click on the top left icon in the editor and create a new buffer.

SOURCE CODE:

```
import java.util.*;

class Student{}

class Rockstar{  }

class Hacker{}

public class InstanceOFTutorial{

  static String count(ArrayList mylist){

    int a = 0,b = 0,c = 0;

    for(int i = 0; i < mylist.size(); i++){

      Object element=mylist.get(i);

      if(element instanceof Student)

        a++;

      if(element instanceof Rockstar)

        b++;

      if(element instanceof Hacker)
```

```
            c++;
        }
        String ret = Integer.toString(a)+" "+ Integer.toString(b)+" "+ Integer.toString(c);
        return ret;
    }
    public static void main(String []args){
        ArrayList mylist = new ArrayList();
        Scanner sc = new Scanner(System.in);
        int t = sc.nextInt();
        for(int i=0; i<t; i++){
            String s=sc.next();
            if(s.equals("Student"))mylist.add(new Student());
            if(s.equals("Rockstar"))mylist.add(new Rockstar());
            if(s.equals("Hacker"))mylist.add(new Hacker());
        }
        System.out.println(count(mylist));
    }
}
```

OUTPUT:

```
5
Student
Student
Rockstar
Student
Hacker
```

Your Output (stdout)

```
3 1 1
```

Expected Output Download

```
3 1 1
```

3.PROBLEM STATEMENT:

An underground railway system is keeping track of customer travel times between different stations. They are using this data to calculate the average time it takes to travel from one station to another.

Implement the UndergroundSystem class:

- void checkIn(int id, string stationName, int t)
    - A customer with a card ID equal to id, checks in at the station stationName at time t.
    - A customer can only be checked into one place at a time.
- void checkOut(int id, string stationName, int t)
    - A customer with a card ID equal to id, checks out from the station stationName at time t.
- double getAverageTime(string startStation, string endStation)
    - Returns the average time it takes to travel from startStation to endStation.
    - The average time is computed from all the previous traveling times from startStation to endStation that happened **directly**, meaning a check in at startStation followed by a check out from endStation.
    - The time it takes to travel from startStation to endStation **may be different** from the time it takes to travel from endStation to startStation.
    - There will be at least one customer that has traveled from startStation to endStation before getAverageTime is called.

You may assume all calls to the checkIn and checkOut methods are consistent. If a customer checks in at time t_1 then checks out at time t_2, then $t_1 < t_2$. All events happen in chronological order.

Constraints:

- $1 <= id, t <= 10^6$
- 1 <= stationName.length, startStation.length, endStation.length <= 10
- All strings consist of uppercase and lowercase English letters and digits.
- There will be at most $2 * 10^4$ calls **in total** to checkIn, checkOut, and getAverageTime.
- Answers within 10^{-5} of the actual value will be accepted.

SOURCE CODE:

```
import java.util.*;
class UndergroundSystem {
    HashMap<Integer, CheckInData> checkIns;
    HashMap<String, RouteData> routes;
    public UndergroundSystem() {
        checkIns = new HashMap<>();
        routes = new HashMap<>();
    }
    public void checkIn(int id, String stationName, int t) {
        checkIns.put(id, new CheckInData(stationName, t));
    }
    public void checkOut(int id, String stationName, int t) {
        CheckInData data = checkIns.get(id);
        String startStation = data.stationName;
        int startTime = data.time;
        int travelTime = t - startTime;
        String route = startStation + "->" + stationName;
        if (!routes.containsKey(route)) {
            routes.put(route, new RouteData());
        }
        RouteData routeData = routes.get(route);
        routeData.totalTime += travelTime;
        routeData.tripCount++;
        checkIns.remove(id);
    }
    public double getAverageTime(String startStation, String endStation) {
      String route = startStation + "->" + endStation;
```

```
        RouteData routeData = routes.get(route);
        return (double) routeData.totalTime / routeData.tripCount;
    }
}
class CheckInData {
    String stationName;
    int time;
    CheckInData(String stationName, int time) {
        this.stationName = stationName;
        this.time = time;
    }
}

class RouteData {
    int totalTime = 0;
    int tripCount = 0;
}
```

OUTPUT:



4.PROBLEM STATEMENT:

You have a browser of one tab where you start on the homepage and you can visit another url, get back in the history number of steps or move forward in the history number of steps.

Implement the BrowserHistory class:

- BrowserHistory(string homepage) Initializes the object with the homepage of the browser.
- void visit(string url) Visits url from the current page. It clears up all the forward history.
- string back(int steps) Move steps back in history. If you can only return x steps in the history and steps > x, you will return only x steps. Return the current url after moving back in history **at most** steps.
- string forward(int steps) Move steps forward in history. If you can only forward x steps in the history and steps > x, you will forward only x steps. Return the current url after forwarding in history **at most** steps.

Constraints:

- 1 <= homepage.length <= 20
- 1 <= url.length <= 20
- 1 <= steps <= 100
- homepage and url consist of '.' or lower case English letters.
- At most 5000 calls will be made to visit, back, and forward.

SOURCE CODE:

```
import java.util.*;

class BrowserHistory {

   ArrayList<String> history;

   int current;

   public BrowserHistory(String homepage) {

      history = new ArrayList<>();

      history.add(homepage);
```

```
        current = 0;
    }
    public void visit(String url) {
        while (history.size() > current + 1) {
            history.remove(history.size() - 1);
        }
        history.add(url);
        // Move current position
        current++;
    }
    public String back(int steps) {
        current = Math.max(0, current - steps);
        return history.get(current);
    }
    public String forward(int steps) {
        current = Math.min(history.size() - 1, current + steps);
        return history.get(current);
    }
}
```

OUTPUT:



5.PROBLEM STATEMENT:

Design a HashSet without using any built-in hash table libraries.

Implement MyHashSet class:

- void add(key) Inserts the value key into the HashSet.
- bool contains(key) Returns whether the value key exists in the HashSet or not.
- void remove(key) Removes the value key in the HashSet. If key does not exist in the HashSet, do nothing.

Constraints:

- $0 <= key <= 10^6$
- At most 10^4 calls will be made to add, remove, and contains.

SOURCE CODE:

```
class MyHashSet {
    boolean[] set;
    // Constructor
    public MyHashSet() {
        set = new boolean[1000001];
    }
    // Add key
    public void add(int key) {
        set[key] = true;
    }
    // Check whether key exists
    public boolean contains(int key) {
        return set[key];
    }

    // Remove key
```

```
    public void remove(int key) {
        set[key] = false;
    }
}
```

Input:

["MyHashSet","add","add","add","remove","contains","add","add","add","remove","contains","add","add","add","remove","contains","add","add","add","remove","contains"]

[[],[1],[10001],[1],[1],[1],[7],[10007],[7],[7],[7],[123],[10123],[123],[123],[123],[5000],[15000],[5000],[5000],[5000]]

Output:

[null,null,null,null,null,false,null,null,null,null,false,null,null,null,null,false,null,null,null,null,false]

6.PROBLEM STATEMENT:

HackerLand University has the following grading policy:

- Every student receives a in the inclusive range from to .
- Any less than is a failing grade.

Sam is a professor at the university and likes to round each student's according to these rules:

- If the difference between the and the next multiple of is less than , round up to the next multiple of .
- If the value of is less than , no rounding occurs as the result will still be a failing grade.

Examples

- round to (85 - 84 is less than 3)
- do not round (result is less than 38)
- do not round (60 - 57 is 3 or higher)

Given the initial value of for each of Sam's students, write code to automate the rounding process.

Function Description

Complete the function with the following parameter(s):

- : the grades before rounding

Returns

- : the grades after rounding

Input Format

The first line contains a single integer, , the number of students.
Each line of the subsequent lines contains a single integer, .

Constraints

-
-

Sample Input 0

4

73

67

38

33

Sample Output 0

75

67

40

33

Explanation 0

ID	Original Grade	Final Grade
1	73	75
2	67	67
3	38	40
4	33	33

1. Student received a , and the next multiple of from is . Since , the student's grade is rounded to .
2. Student received a , and the next multiple of from is . Since , the grade will not be modified and the student's final grade is .
3. Student received a , and the next multiple of from is . Since , the student's grade will be rounded to .
4. Student received a grade below , so the grade will not be modified and the student's final grade is 33.

Source code:

```
import java.io.*;

import java.util.*;


public class Solution {


    public static List<Integer> gradingStudents(List<Integer> grades) {
```

```
    List<Integer> result = new ArrayList<>();

    for (int grade : grades) {

        // Grades below 38 are not rounded
        if (grade < 38) {
            result.add(grade);
        }
        else {
            // Find the next multiple of 5
            int nextMultiple = ((grade / 5) + 1) * 5;

            // Round if difference is less than 3
            if (nextMultiple - grade < 3) {
                result.add(nextMultiple);
            }
            else {
                result.add(grade);
            }
        }
    }

    return result;
}

public static void main(String[] args) throws IOException {

    BufferedReader br = new BufferedReader(
```

```
            new InputStreamReader(System.in)
        );

        int n = Integer.parseInt(br.readLine());

        List<Integer> grades = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            grades.add(Integer.parseInt(br.readLine()));
        }
        List<Integer> result = gradingStudents(grades);

        for (int grade : result) {
            System.out.println(grade);
        }
    }
}
```

Input :

```
4
 73
67
38
33
```

Output :

```
75
67
 40
```

7.PROBLEM STATEMENT:

Using *inheritance*, one class can acquire the properties of others. Consider the following *Animal* class:

```
class Animal{
   void walk(){
      System.out.println("I am walking");
   }
}
```

This class has only one method, *walk*. Next, we want to create a *Bird* class that also has a *fly* method. We do this using *extends* keyword:

```
class Bird extends Animal {
   void fly() {
      System.out.println("I am flying");
   }
}
```

Finally, we can create a Bird object that can both *fly* and *walk*.

```
public class Solution{
  public static void main(String[] args){
     Bird bird = new Bird();
     bird.walk();
     bird.fly();
  }
}
```

The above code will print:

I am walking

I am flying

This means that a Bird object has all the properties that an Animal object has, as well as some additional unique properties.

The code above is provided for you in your editor. You must add a *sing* method to the *Bird* class, then modify the *main* method accordingly so that the code prints the following lines:

I am walking

I am flying

I am singing

Source code:

```
class Animal {
    void walk() {
        System.out.println("I am walking");
    }
}
class Bird extends Animal {
    void fly() {
        System.out.println("I am flying");
    }
    void sing() {
        System.out.println("I am singing");
    }
}
public class Solution {
    public static void main(String[] args) {
        Bird bird = new Bird();
        bird.walk();
        bird.fly();
        bird.sing();
    }
}
```

Output :

```
I am walking
I am flying
I am sing
```

8.PROBLEM STATEMENT:

Write the following code in your editor below:

1. A class named *Arithmetic* with a method named *add* that takes 2 integers as parameters and returns an integer denoting their sum.
2. A class named *Adder* that inherits from a superclass named *Arithmetic*.

Your classes should not be be public

Input Format

You are not responsible for reading any input from stdin; a locked code stub will test your submission by calling the *add* method on an *Adder* object and passing it 2 integer parameters.

Output Format

You are not responsible for printing anything to stdout. Your *add* method must return the sum of its parameters.

Sample Output

The *main* method in the *Solution* class above should print the following:

My superclass is: Arithmetic

42 13 20

Source code:

```
class Arithmetic {


   int add(int a, int b) {
      return a + b;
   }
}
class Adder extends Arithmetic {


}
```

Output:

My superclass is: Arithmetic

42 13 20

9.PROBLEM STATEMENT:

Generic methods are a very efficient way to handle multiple datatypes using a single method. This problem will test your knowledge on Java Generic methods.

Let's say you have an integer array and a string array. You have to write a **single** method *printArray* that can print all the elements of both arrays. The method should be able to accept both integer arrays or string arrays.

You are given code in the editor. Complete the code so that it prints the following lines:

1

2

3

Hello

World

Do not use method overloading because your answer will not be accepted.

Source code:

```
import java.io.*;
import java.util.*;


public class Solution {


   public static <T> void printArray(T[] array) {


      for (T element : array) {
         System.out.println(element);
      }
   }


   public static void main(String[] args) {


      Integer[] intArray = {1, 2, 3};
      String[] stringArray = {"Hello", "World"};
```

```
        printArray(intArray);
        printArray(stringArray);
    }
}
```

Output:

```
1
2
3
Hello
World
```

10.PROBLEM STATEMENT:

A kingdom consists of a king, his children, his grandchildren, and so on. Every once in a while, someone in the family dies or a child is born.

The kingdom has a well-defined order of inheritance that consists of the king as the first member. Let's define the recursive function Successor(x, curOrder), which given a person x and the inheritance order so far, returns who should be the next person after x in the order of inheritance.

Successor(x, curOrder):

if x has no children or all of x's children are in curOrder:

if x is the king return null

else return Successor(x's parent, curOrder)

else return x's oldest child who's not in curOrder

For example, assume we have a kingdom that consists of the king, his children Alice and Bob (Alice is older than Bob), and finally Alice's son Jack.

1. In the beginning, curOrder will be ["king"].
2. Calling Successor(king, curOrder) will return Alice, so we append to curOrder to get ["king", "Alice"].
3. Calling Successor(Alice, curOrder) will return Jack, so we append to curOrder to get ["king", "Alice", "Jack"].
4. Calling Successor(Jack, curOrder) will return Bob, so we append to curOrder to get ["king", "Alice", "Jack", "Bob"].
5. Calling Successor(Bob, curOrder) will return null. Thus the order of inheritance will be ["king", "Alice", "Jack", "Bob"].

Using the above function, we can always obtain a unique order of inheritance.

Implement the ThroneInheritance class:

- ThroneInheritance(string kingName) Initializes an object of the ThroneInheritance class. The name of the king is given as part of the constructor.
- void birth(string parentName, string childName) Indicates that parentName gave birth to childName.
- void death(string name) Indicates the death of name. The death of the person doesn't affect the Successor function nor the current inheritance order. You can treat it as just marking the person as dead.
- string[] getInheritanceOrder() Returns a list representing the current order of inheritance **excluding** dead people.

Example 1:

Input

["ThroneInheritance", "birth", "birth", "birth", "birth", "birth", "birth", "getInheritanceOrder", "death", "getInheritanceOrder"]

[["king"], ["king", "andy"], ["king", "bob"], ["king", "catherine"], ["andy", "matthew"], ["bob", "alex"], ["bob", "asha"], [null], ["bob"], [null]]

Output

[null, null, null, null, null, null, null, ["king", "andy", "matthew", "bob", "alex", "asha", "catherine"], null, ["king", "andy", "matthew", "alex", "asha", "catherine"]]

Explanation

ThroneInheritance t= new ThroneInheritance("king"); // order: **king**

t.birth("king", "andy"); // order: king > **andy**

t.birth("king", "bob"); // order: king > andy > **bob**

t.birth("king", "catherine"); // order: king > andy > bob > **catherine**

t.birth("andy", "matthew"); // order: king > andy > **matthew** > bob > catherine

t.birth("bob", "alex"); // order: king > andy > matthew > bob > **alex** > catherine

t.birth("bob", "asha"); // order: king > andy > matthew > bob > alex > **asha** > catherine

t.getInheritanceOrder(); // return ["king", "andy", "matthew", "bob", "alex", "asha", "catherine"]

t.death("bob"); // order: king > andy > matthew > **bob** > alex > asha > catherine

t.getInheritanceOrder(); // return ["king", "andy", "matthew", "alex", "asha", "catherine"]

Constraints:

- 1 <= kingName.length, parentName.length, childName.length, name.length <= 15
- kingName, parentName, childName, and name consist of lowercase English letters only.
- All arguments childName and kingName are **distinct**.
- All name arguments of death will be passed to either the constructor or as childName to birth first.

- For each call to birth(parentName, childName), it is guaranteed that parentName is alive.
- At most 10^5 calls will be made to birth and death.
- At most 10 calls will be made to getInheritanceOrder.

Source code:

```
import java.util.*;


class ThroneInheritance {

    // Stores children of each person
    HashMap<String, List<String>> family;

    // Stores dead people
    HashSet<String> dead;

    String king;

    // Constructor
    public ThroneInheritance(String kingName) {
        king = kingName;
        family = new HashMap<>();
        dead = new HashSet<>();

        family.put(kingName, new ArrayList<>());
    }

    // Add a child
    public void birth(String parentName, String childName) {
```

```
    if (!family.containsKey(parentName)) {
        family.put(parentName, new ArrayList<>());
    }

    family.get(parentName).add(childName);

    // Initialize child's children list
    family.put(childName, new ArrayList<>());
}

// Mark a person as dead
public void death(String name) {
    dead.add(name);
}

// Get current inheritance order
public List<String> getInheritanceOrder() {

    List<String> result = new ArrayList<>();

    dfs(king, result);

    return result;
}

// Depth First Search
private void dfs(String person, List<String> result) {

    // Add person if alive
```

```
        if (!dead.contains(person)) {
            result.add(person);
        }

        // Visit children from oldest to youngest
        for (String child : family.get(person)) {
            dfs(child, result);
        }
    }
}
```

Input:

["ThroneInheritance","birth","birth","birth","birth","birth","birth","getInheritanceOrder","death","getInheritanceOrder"]

[["king"],["king","andy"],["king","bob"],["king","catherine"],["andy","matthew"],["bob","alex"],["bob","asha"],[null],["bob"],[null]]

Output:

[null,null,null,null,null,null,null,["king","andy","matthew","bob","alex","asha","catherine"],null,["king","andy","matthew","alex","asha","catherine"]]

WEEK-8

1.PROBLEM STATEMENT:

When a subclass inherits from a superclass, it inherits its methods; however, it can also override superclass methods to customize their behavior.

SOURCE CODE:

```
import java.util.*;
class Sports{
   String getName(){
      return "Generic Sports";
   }
   void getNumberOfTeamMembers(){
      System.out.println( "Each team has n players in " + getName() );
   }
}
class Soccer extends Sports{
   @Override
   String getName(){
      return "Soccer Class";
   }
class Sports {
   String getName() {
      return "Generic Sports";
   }
   void getNumberOfTeamMembers() {
      System.out.println("Each team has n players in " + getName());
   }
}
   @Override
   void getNumberOfTeamMembers() {
```

```
        System.out.println("Each team has 11 players in " + getName());
}
}
public class Solution{
    public static void main(String []args){
        Sports c1 = new Sports();
        Soccer c2 = new Soccer();
        System.out.println(c1.getName());
        c1.getNumberOfTeamMembers();
        System.out.println(c2.getName());
        c2.getNumberOfTeamMembers();
    }
}
```

OUTPUT:

Your Output (stdout)

```
Generic Sports
Each team has n players in Generic Sports
Soccer Class
Each team has 11 players in Soccer Class
```

Expected Output Download

```
Generic Sports
Each team has n players in Generic Sports
Soccer Class
Each team has 11 players in Soccer
```

2.PROBLEM STAEMENT:

When a method in a subclass overrides a method in a superclass, it is still possible to call the overridden method using the super keyword. Calling super.func() invokes the version of func() defined in the superclass.

You are given a partially completed code in the editor. Modify the code so that it prints the following text:

SOURCE CODE:

```
import java.util.*;
import java.io.*;

class BiCycle{
   String define_me(){
      return "a vehicle with pedals.";
   }
}

class MotorCycle extends BiCycle{
   String define_me(){
      return "a cycle with an engine.";
   }

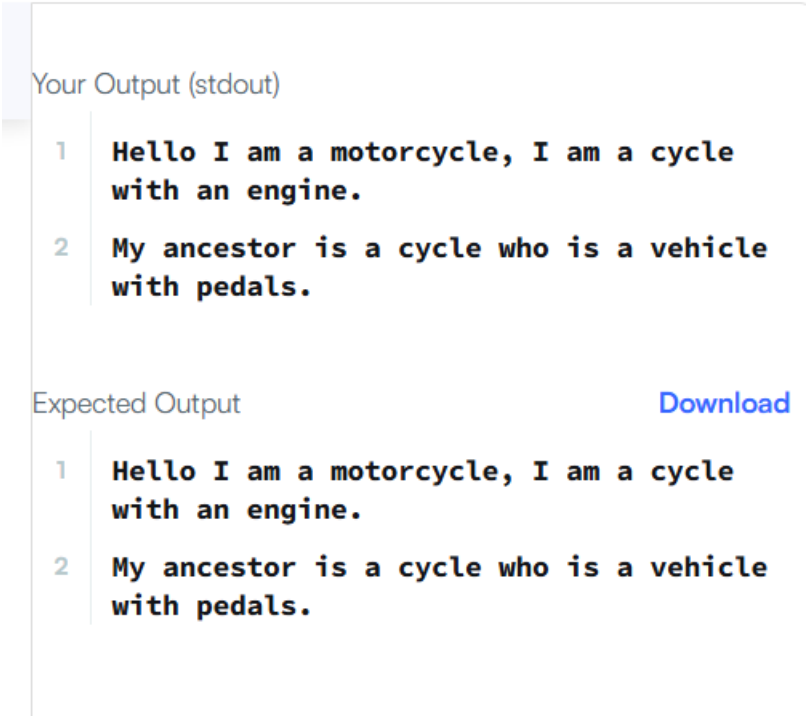
   MotorCycle(){
      System.out.println("Hello I am a motorcycle, I am "+ define_me());

      String temp = super.define_me(); // Fix this line

      System.out.println("My ancestor is a cycle who is "+ temp );
   }
```

```
}
class Solution{
    public static void main(String []args){
        MotorCycle M=new MotorCycle();
    }
}
```

OUTPUT:



3.PROBLEM STATEMENT:

“The singleton pattern is a design pattern that restricts the instantiation of a class to one object. This is useful when exactly one object is needed to coordinate actions across the system." — *Wikipedia: Singleton Pattern*

Complete the Singleton class in your editor, which must contain the following components:

1. A private Singleton non-parameterized constructor.
2. A public String instance variable named str.
3. A static method named getSingleInstance that returns the single instance of the Singleton class.

SOURCE CODE:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;
import java.lang.reflect.*;

class Singleton{
private static Singleton instance;

   // Public String instance variable named str
   public String str;

   // Private constructor to prevent instantiation from outside
   private Singleton() {
   }
```

```
    // Static method to return the single instance of the Singleton class
    public static Singleton getSingleInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}

public class Main {

 public static void main(String args[])throws Exception{

    Scanner sc=new Scanner(System.in);
    Singleton s1 = Singleton.getSingleInstance(); //retrive the single instance
    Singleton s2=Singleton.getSingleInstance();
    assert(s1==s2);

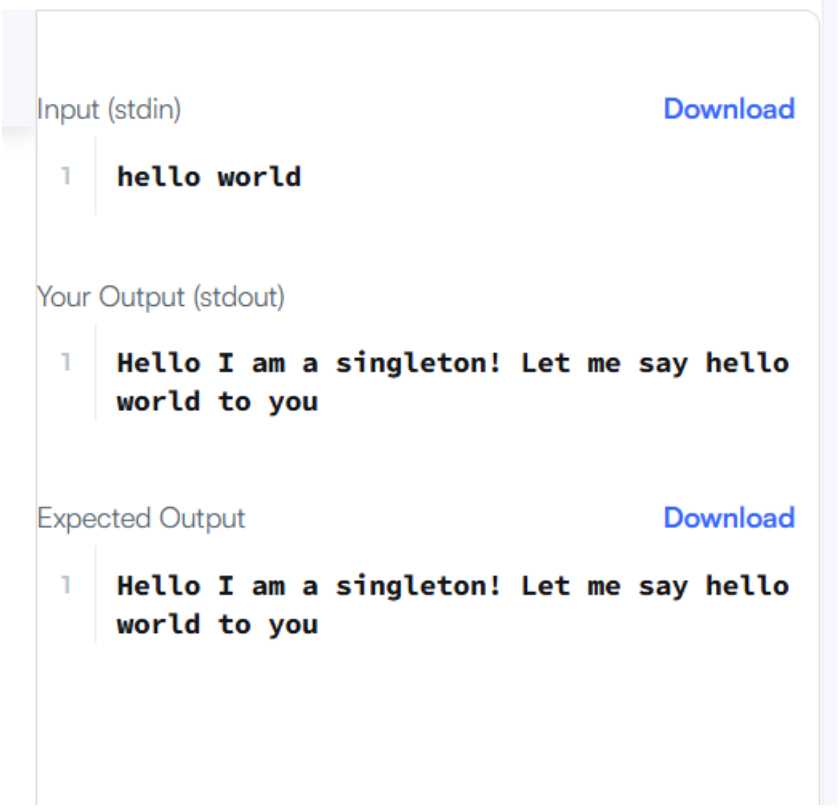
    //verify that the constructor is private
    Class c=s1.getClass();
    Constructor[] allConstructors = c.getDeclaredConstructors();
    assert allConstructors.length==1;
    for (Constructor ctor : allConstructors)
    {
        if(ctor.getModifiers()!=2 || !ctor.toString().equals("private Singleton()")) //The
constructor must be private
        {
            System.out.println("Wrong class!");
        }
```

```
    }
    String str=sc.nextLine();
    s1.str=str;
    s2.str=str;
    System.out.println("Hello I am a singleton! Let me say "+str+" to you");

 }

}
```

OUTPUT:



4.PROBLEM STATEMENT:

An important concept in Object-Oriented Programming is the open/closed principle, which means writing code that is open to extension but closed to modification. This challenge simulates a real-life problem where the open/closed principle should be applied using the **Visitor Design Pattern**.

A Tree class implementing a rooted tree is provided in the uneditable template. It has the following public methods:

- getValue(): Returns the value stored in the node.
- getColor(): Returns the color of the node (Color.RED or Color.GREEN).
- getDepth(): Returns the depth of the node (the root node has depth 0).

SOURCE CODE:

```
import java.util.*;

// 1. Implement SumInLeavesVisitor

class SumInLeavesVisitor extends TreeVis {

    private int result = 0;

    public int getResult() {

        return result;

    }

    public void visitNode(TreeNode node) {

        // Non-leaf nodes are ignored for this visitor

    }

    public void visitLeaf(TreeLeaf leaf) {

        result += leaf.getValue();

    }

}

// 2. Implement ProductRedNodesVisitor

class ProductRedNodesVisitor extends TreeVis {

    private long result = 1;

    private final int MOD = 1000000007;
```

```
    public int getResult() {
        return (int) result;
    }
    public void visitNode(TreeNode node) {
        if (node.getColor() == Color.RED) {
            result = (result * node.getValue()) % MOD;
        }
    }
    public void visitLeaf(TreeLeaf leaf) {
        if (leaf.getColor() == Color.RED) {
            result = (result * leaf.getValue()) % MOD;
        }
    }
}
// 3. Implement FancyVisitor
class FancyVisitor extends TreeVis {
    private int nonLeafEvenDepthSum = 0;
    private int greenLeafSum = 0;
    public int getResult() {
        return Math.abs(nonLeafEvenDepthSum - greenLeafSum);
    }
    public void visitNode(TreeNode node) {
        if (node.getDepth() % 2 == 0) {
            nonLeafEvenDepthSum += node.getValue();
        }
    }
    public void visitLeaf(TreeLeaf leaf) {
        if (leaf.getColor() == Color.GREEN) {
```

```
            greenLeafSum += leaf.getValue();
        }
    }
}
// 4. Implement tree construction logic inside Solution class
public class Solution {
    private static int[] values;
    private static Color[] colors;
    private static Map<Integer, List<Integer>> nodeMap = new HashMap<>();
    public static Tree solve() {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        values = new int[n];
        for (int i = 0; i < n; i++) {
            values[i] = sc.nextInt();
        }
        colors = new Color[n];
        for (int i = 0; i < n; i++) {
            colors[i] = (sc.nextInt() == 0) ? Color.RED : Color.GREEN;
        }
        for (int i = 0; i < n - 1; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();
            nodeMap.computeIfAbsent(u, k -> new ArrayList<>()).add(v);
            nodeMap.computeIfAbsent(v, k -> new ArrayList<>()).add(u);
        }
        sc.close();

        // Edge case: single-node tree is a leaf
```

```
        if (n == 1) {
            return new TreeLeaf(values[0], colors[0], 0);
        }

        TreeNode root = new TreeNode(values[0], colors[0], 0);
        buildTree(root, 1);

        return root;
    }

    private static void buildTree(TreeNode parent, int parentNum) {
        List<Integer> neighbors = nodeMap.get(parentNum);
        if (neighbors == null) return;

        for (int childNum : neighbors) {
            // Remove edge pointing back to parent to avoid infinite loop
            nodeMap.get(childNum).remove(Integer.valueOf(parentNum));

            List<Integer> grandChildren = nodeMap.get(childNum);
            boolean isLeaf = (grandChildren == null || grandChildren.isEmpty());

            Tree childNode;
            if (isLeaf) {
                childNode = new TreeLeaf(values[childNum - 1], colors[childNum - 1],
parent.getDepth() + 1);
            } else {
                childNode = new TreeNode(values[childNum - 1], colors[childNum - 1],
parent.getDepth() + 1);
                buildTree((TreeNode) childNode, childNum);
            }
```

```
            parent.addChild(childNode);
        }
    }
}
```

OUTPUT:

Sample Input

```
5
4 7 2 5 12
0 1 0 0 1
1 2
1 3
3 4
3 5
```

Sample Output

```
24
40
15
```

5.PROBLEM STATEMENT:

Java Iterator class can help you to iterate through every element in a collection. Here is a simple example:

SOURCE CODE:

```
import java.util.*;
public class Main{


  static Iterator func(ArrayList mylist){
    Iterator it=mylist.iterator();
    while(it.hasNext()){
      Object element = it.next();
      if (element instanceof String)


        break;
     }
    return it;


  }
  @SuppressWarnings({ "unchecked" })
  public static void main(String []args){
    ArrayList mylist = new ArrayList();
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    int m = sc.nextInt();
    for(int i = 0;i<n;i++){
      mylist.add(sc.nextInt());
    }


    mylist.add("###");
```

```
    for(int i=0;i<m;i++){
      mylist.add(sc.next());
    }

    Iterator it=func(mylist);
    while(it.hasNext()){
      Object element = it.next();
      System.out.println((String)element);
    }
  }
}
```

OUTPUT:



6.PROBLEM STATEMENT:

A Java interface can only contain method signatures and fields. Interface methods are abstract by default (unless defined as default or static in modern Java), allowing you to achieve polymorphism. In this problem, you will practice your knowledge of interfaces.

You are given an interface AdvancedArithmetic which contains a single method signature:

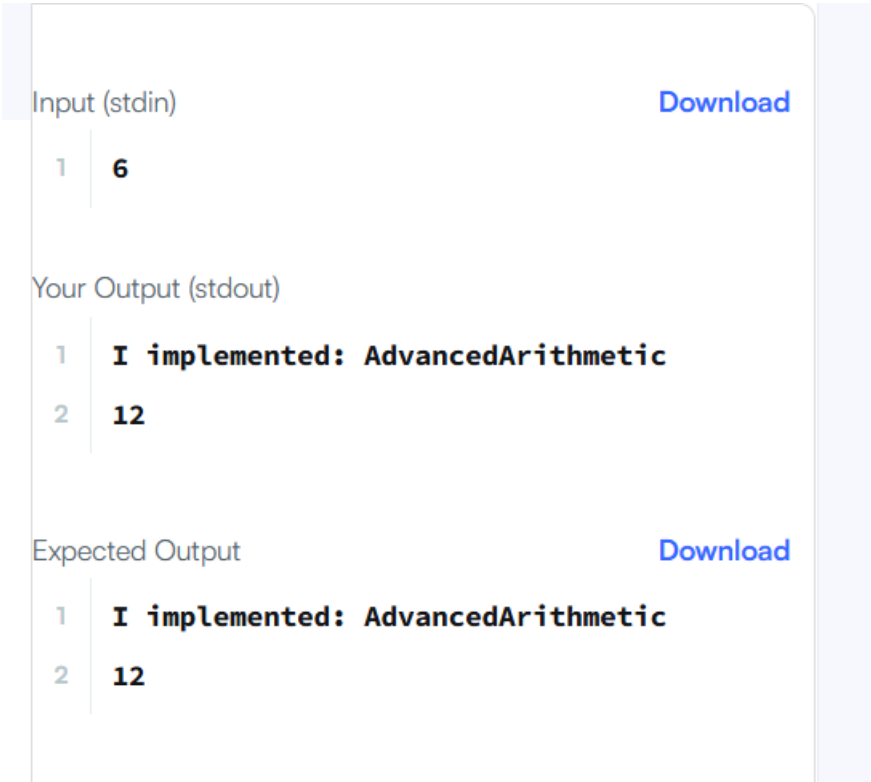
SOURCE CODE:

```
import java.util.*;

interface AdvancedArithmetic{
  int divisor_sum(int n);
}
class MyCalculator implements AdvancedArithmetic {
    public int divisor_sum(int n) {
        int sum = 0;
        for (int i = 1; i <= n; i++) {
            if (n % i == 0) {
                sum += i;
            }
        }
        return sum;
    }
}
class Solution{
    public static void main(String []args){
        MyCalculator my_calculator = new MyCalculator();
        System.out.print("I implemented: ");
        ImplementedInterfaceNames(my_calculator);
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
```

```
        System.out.print(my_calculator.divisor_sum(n) + "\n");
        sc.close();
    }
    /*
     * ImplementedInterfaceNames method takes an object and prints the name of the interfaces it implemented
     */
    static void ImplementedInterfaceNames(Object o){
        Class[] theInterfaces = o.getClass().getInterfaces();
        for (int i = 0; i < theInterfaces.length; i++){
            String interfaceName = theInterfaces[i].getName();
            System.out.println(interfaceName);
        }
    }
}
```

OUTPUT:



7.PROBLEM STATEMENT:

Using inheritance, one class can acquire the properties of others. Consider the following

SOURCE CODE:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

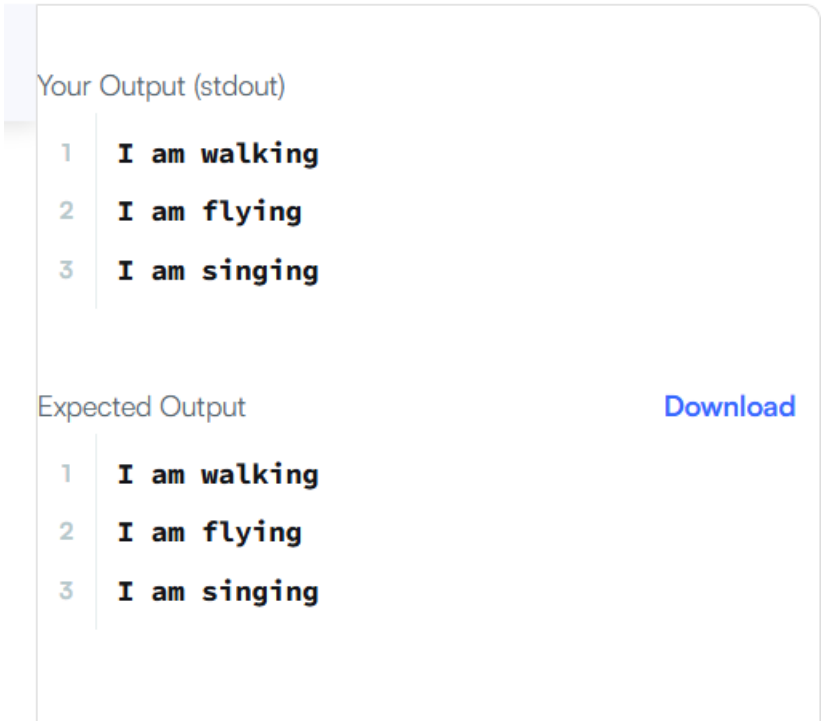

class Animal{
   void walk()
   {
      System.out.println("I am walking");
   }
}
class Bird extends Animal
{
   void fly()
   {
      System.out.println("I am flying");
   }
   void sing() {
      System.out.println("I am singing");
   }
}
public class Solution{


  public static void main(String args[]){
```

```
        Bird bird = new Bird();
        bird.walk();
        bird.fly();
        bird.sing();

    }
}
```

OUTPUT:



8.PROBLEM STATEMENT:

Write the following classes in your editor:

1. A class named Arithmetic with a method named add that takes two integers as parameters and returns an integer denoting their sum.
2. A class named Adder that inherits from a superclass named Arithmetic.

Task Requirements

- You do not need to read any input from standard input (stdin).
- A locked code stub in the editor tests your solution by creating an instance of Adder, calling its add method with integer arguments, and checking its superclass structure.

SOURCE CODE:

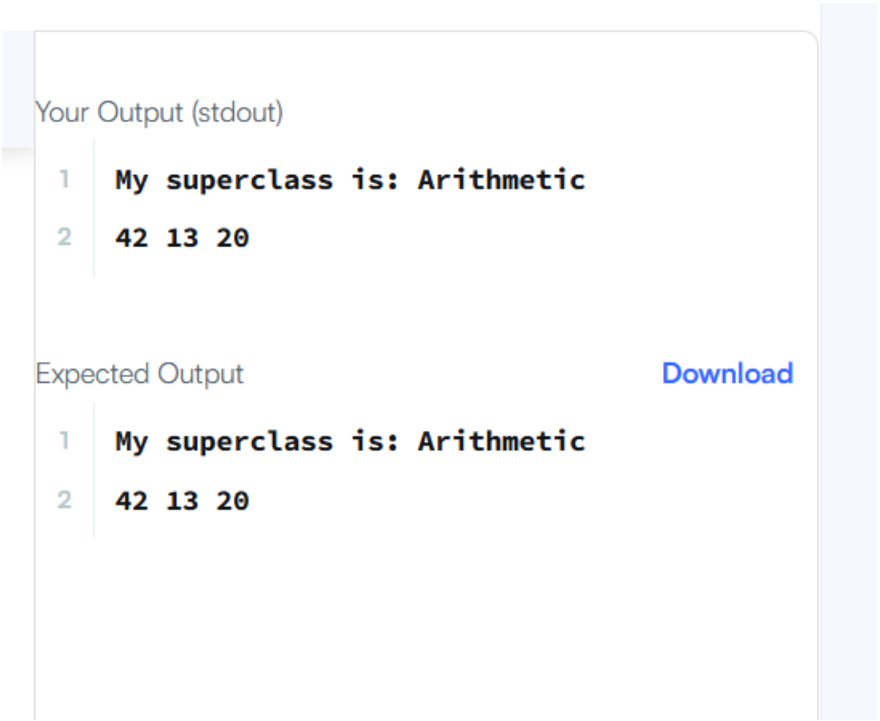
```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;
class Arithmetic {
   public int add(int a, int b) {
      return a + b;
   }
}

class Adder extends Arithmetic {
}
public class Solution{
   public static void main(String []args){
      // Create a new Adder object
      Adder a = new Adder();
```

```
        // Print the name of the superclass on a new line
        System.out.println("My superclass is: " + a.getClass().getSuperclass().getName());

        // Print the result of 3 calls to Adder's `add(int,int)` method as 3 space-separated integers:
        System.out.print(a.add(10,32) + " " + a.add(10,3) + " " + a.add(10,10) + "\n");
    }
}
```

OUTPUT:



9.PROBLEM STATEMENT:

Generic methods are a very efficient way to handle multiple datatypes using a single method. This problem will test your knowledge on Java Generic methods.

Let's say you have an integer array and a string array. You have to write a **single** method *printArray* that can print all the elements of both arrays. The method should be able to accept both integer arrays or string arrays.

SOURCE CODE:

```
import java.io.*;
import java.util.*;
import java.lang.reflect.Method;
class Printer {
   // Generic method to print array elements of any type
   public <E> void printArray(E[] array) {
      for (E element : array) {
         System.out.println(element);
      }
   }
}
public class Solution {
   public static void main(String[] args) {
      Printer myPrinter = new Printer();
      Integer[] intArray = { 1, 2, 3 };
      String[] stringArray = { "Hello", "World" }
      myPrinter.printArray(intArray);
      myPrinter.printArray(stringArray);
      int count = 0;
      for (Method method : Printer.class.getDeclaredMethods()) {
         if (method.getName().equals("printArray")) {
            count++;
```

```
            }
        }
        if (count > 1) {
            System.out.println("Method overloading is not allowed!");
        }
    }
}
```

OUTPUT:



10.PROBLEM STATEMENT:

A kingdom consists of a king, his children, his grandchildren, and so on. Every once in a while, someone in the family dies or a child is born.

The kingdom has a well-defined order of inheritance that consists of the king as the first member. Let's define the recursive function Successor(x, curOrder), which given a person x and the inheritance order so far, returns who should be the next person after x in the order of inheritance.

Successor(x, curOrder):

if x has no children or all of x's children are in curOrder:

if x is the king return null

else return Successor(x's parent, curOrder)

else return x's oldest child who's not in curOrder

SOURCE CODE:

```
import java.util.*;


class ThroneInheritance {
    private String king;
    private Map<String, List<String>> children;
    private Set<String> dead;

    public ThroneInheritance(String kingName) {
        this.king = kingName;
        this.children = new HashMap<>();
        this.dead = new HashSet<>();
        this.children.put(kingName, new ArrayList<>());
    }

    public void birth(String parentName, String childName) {
```

```
        children.putIfAbsent(parentName, new ArrayList<>());
        children.get(parentName).add(childName);
    }

    public void death(String name) {
        dead.add(name);
    }

    public List<String> getInheritanceOrder() {
        List<String> order = new ArrayList<>();
        dfs(king, order);
        return order;
    }

    private void dfs(String current, List<String> order) {
        // Add to result if the person is alive
        if (!dead.contains(current)) {
            order.add(current);
        }

        // Recursively visit all children in order of birth
        if (children.containsKey(current)) {
            for (String child : children.get(current)) {
                dfs(child, order);
            }
        }
    }
}
```

OUTPUT:


Testcase
Test Result
Accepted
Runtime: 4 ms
Case 1
Input
["ThroneInheritance","birth","birth","birth","birth","birth","birth","getInheri
nceOrder","death","getInheritanceOrder"]
[["king"],["king","andy"],["king","bob"],["king","catherine"],["andy","matthew"],
["bob","alex"],["bob","asha"],[null],["bob"],[null]]
Output
[null,null,null,null,null,null,null,["king","andy","matthew","bob","alex","asha",
"catherine"],null,["king","andy","matthew","alex","asha","catherine"]]
Expected
[null,null,null,null,null,null,null,["king","andy","matthew","bob","alex","asha",
"catherine"],null,["king","andy","matthew","alex","asha","catherine"]]

WEEK-9

1.PROBLEM STATEMENT:

To write a Java program using functional transformation techniques and streams to increase each employee's salary by 10% and display the updated salaries.

SOURCE CODE:

```
import java.util.*;
import java.util.stream.*;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        List<Integer> salaries = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            salaries.add(sc.nextInt());
        }

        List<Integer> updatedSalaries = salaries.stream()
                .map(salary -> (int)(salary * 1.10))
                .collect(Collectors.toList());

        for (int salary : updatedSalaries) {
            System.out.print(salary + " ");
        }

        sc.close();
    }
```

}

OUTPUT:

```
5
1000 2000 3000 4000 5000
1100 2200 3300 4400 5500
```

2.PROBLEM STATEMENT:

To write a Java program using **Stream API, filtering, grouping, averaging, and sorting** to analyze sensor temperature readings.

SOURCE CODE:

```
package com;

import java.util.*;

import java.util.stream.*;


public class RealTimeStreamAnalytics {


   static class Sensor {
      String id;
      double temp;


      Sensor(String id, double temp) {
         this.id = id;
         this.temp = temp;
      }
   }


   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);


      int n = sc.nextInt();


      List<Sensor> sensors = new ArrayList<>();
```

```
        for (int i = 0; i < n; i++) {
            String id = sc.next();
            double temp = sc.nextDouble();

            sensors.add(new Sensor(id, temp));
        }

        sensors.stream()
                .filter(s -> s.temp > 50)
                .collect(Collectors.groupingBy(
                        s -> s.id,
                        Collectors.averagingDouble(s -> s.temp)
                ))
                .entrySet()
                .stream()
                .sorted((a, b) -> Double.compare(b.getValue(), a.getValue()))
                .forEach(entry ->
                        System.out.println(entry.getKey() + " " + entry.getValue())
                );

        sc.close();
    }
}
```

OUTPUT:

```
6
s1 60
s2 40
s3 80
s2 90
s1 80
s3 20
s2 90.0
s3 80.0
s1 70.0
```

3.PROBLEM STATEMENT:

To write a Java program using **Kadane's Algorithm** to find the maximum possible profit from a contiguous sequence of daily profit/loss values.

SOURCE CODE:

```
package com;

import java.util.*;

public class KadanesAlgorithm {

public static void main(String[] args) {

int arr[]= {-2,1,-3,4,-1,2,1,-5,4};

int currentsum=arr[0];

int maxsum=arr[0];

for(int i=1;i<arr.length;i++) {

currentsum=Math.max(arr[i],arr[i]+currentsum);

maxsum=Math.max(maxsum,currentsum);

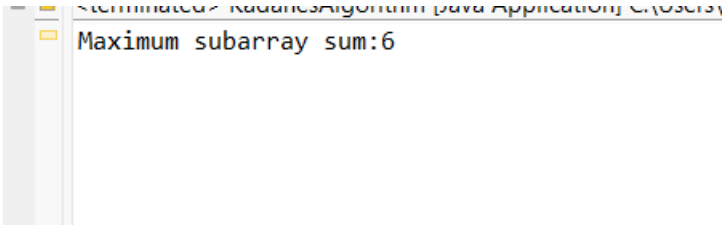
}

System.out.println("Maximum subarray sum:"+maxsum);

}

}
```

OUTPUT:



4.PROBLEM STATEMENT:

To write a Java program using the **KMP (Knuth-Morris-Pratt) algorithm** to efficiently find all occurrences of a DNA pattern in a given DNA sequence.

SOURCE CODE:

```
import java.util.*;


public class IntelligentDNAPatternSearch {


    // Create LPS array
    static int[] computeLPS(String pattern) {
        int[] lps = new int[pattern.length()];


        int len = 0;
        int i = 1;


        while (i < pattern.length()) {
            if (pattern.charAt(i) == pattern.charAt(len)) {
                len++;
                lps[i] = len;
                i++;
            } else {
                if (len != 0) {
                    len = lps[len - 1];
                } else {
                    lps[i] = 0;
                    i++;
                }
            }
        }
```

```
    return lps;
}

// KMP pattern search
static List<Integer> search(String text, String pattern) {
    List<Integer> result = new ArrayList<>();

    int[] lps = computeLPS(pattern);

    int i = 0;
    int j = 0;

    while (i < text.length()) {

        if (text.charAt(i) == pattern.charAt(j)) {
            i++;
            j++;
        }

        if (j == pattern.length()) {
            result.add(i - j);
            j = lps[j - 1];
        } else if (i < text.length()
                && text.charAt(i) != pattern.charAt(j)) {

            if (j != 0) {
                j = lps[j - 1];
            } else {
```

```
                i++;
            }
        }
    }

    return result;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    String text = sc.nextLine();
    String pattern = sc.nextLine();

    List<Integer> positions = search(text, pattern);

    System.out.println(positions.size());

    for (int position : positions) {
        System.out.print(position + " ");
    }

    sc.close();
}
}
```

OUTPUT:

```
AABAACAADAABAABA
AABA
3
0 9 12
```

5.PROBLEM STATEMENT:

To design a Java program using **classes and methods** to perform deposit and withdrawal operations on a bank account and display the final balance

SOURCE CODE:

```
import java.util.*;


class BankAccount {
    int balance = 0;


    void deposit(int amount) {
        balance += amount;
    }


    void withdraw(int amount) {
        balance -= amount;
    }


    int getBalance() {
        return balance;
    }
}


public class BankingTransactionSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);


        int n = sc.nextInt();
        BankAccount account = new BankAccount();
```

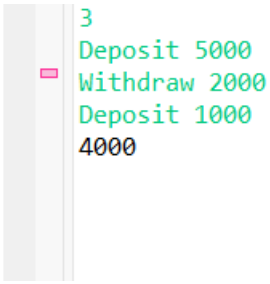
```
        for (int i = 0; i < n; i++) {
            String operation = sc.next();
            int amount = sc.nextInt();

            if (operation.equals("Deposit")) {
                account.deposit(amount);
            } else if (operation.equals("Withdraw")) {
                account.withdraw(amount);
            }
        }

        System.out.println(account.getBalance());

        sc.close();
    }
}
```

OUTPUT:



6.PROBLEM STATEMENT:

To design an **object-oriented ride-sharing system** using reusable classes such as Driver, Rider, Vehicle, and Trip, with **polymorphic fare calculation** for Bike, Auto, and Cab rides and exception handling for invalid bookings.

SOURCE CODE:

```
import java.util.*;
class Driver {
    String name;
    Driver(String name) {
        this.name = name;
    }
}
class Rider {
    String name;
    Rider(String name) {
        this.name = name;
    }
}
abstract class Vehicle {
    abstract int calculateFare(int distance);
}
class Bike extends Vehicle {
    int calculateFare(int distance) {
        return distance * 5;
    }
}
class Auto extends Vehicle {
    int calculateFare(int distance) {
        return distance * 12;
```

```
    }
}
class Cab extends Vehicle {
    int calculateFare(int distance) {
        return distance * 12;
    }
}

class InvalidBookingException extends Exception {
    InvalidBookingException(String message) {
        super(message);
    }
}
class Trip {
    Driver driver;
    Rider rider;
    Vehicle vehicle;
    int distance;
    Trip(Driver driver, Rider rider, Vehicle vehicle, int distance)
            throws InvalidBookingException {
        if (vehicle == null || distance <= 0) {
            throw new InvalidBookingException("Invalid Booking");
        }
        this.driver = driver;
        this.rider = rider;
        this.vehicle = vehicle;
        this.distance = distance;
    }
```

```
    int getFare() {
        return vehicle.calculateFare(distance);
    }
}
public class RideSharingPlatformSimulator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        Driver driver = new Driver("Driver");
        Rider rider = new Rider("Rider");
        for (int i = 0; i < n; i++) {
            String type = sc.next();
            int distance = sc.nextInt();
            Vehicle vehicle = null;
            if (type.equalsIgnoreCase("Bike")) {
                vehicle = new Bike();
            } else if (type.equalsIgnoreCase("Auto")) {
                vehicle = new Auto();
            } else if (type.equalsIgnoreCase("Cab")) {
                vehicle = new Cab();
            }
            try {
                Trip trip = new Trip(driver, rider, vehicle, distance);
                System.out.println(trip.getFare());
            } catch (InvalidBookingException e) {
                System.out.println(e.getMessage());
            }
        }
```

```
        sc.close();
    }
}
```

SAMPLE INPUT:

```
4
Bike 10
Cab 15
Auto 8
Bike 0
```

Output:

```
Plain text

50
180
96
Invalid Booking
```

7.PROBLEM STAEMENT:

To design a **Vehicle Rental System** using **Java inheritance and runtime polymorphism**, where Car, Bike, and Truck calculate rental amounts differently by overriding the calculateRent() method.

SOURCE CODE:

```
import java.util.*;


class Vehicle {
    String vehicleNumber;
    double rentPerDay;


    Vehicle(String vehicleNumber, double rentPerDay) {
        this.vehicleNumber = vehicleNumber;
        this.rentPerDay = rentPerDay;
    }


    double calculateRent(int days) {
        return rentPerDay * days;
    }
```

```
}

class Car extends Vehicle {

    Car(String vehicleNumber, double rentPerDay) {
        super(vehicleNumber, rentPerDay);
    }

    @Override
    double calculateRent(int days) {
        return rentPerDay * days;
    }
}

class Bike extends Vehicle {

    Bike(String vehicleNumber, double rentPerDay) {
        super(vehicleNumber, rentPerDay);
    }

    @Override
    double calculateRent(int days) {
        return rentPerDay * days * 0.90;
    }
}

class Truck extends Vehicle {

    Truck(String vehicleNumber, double rentPerDay) {
```

```
        super(vehicleNumber, rentPerDay);
    }

    @Override
    double calculateRent(int days) {
        return rentPerDay * days * 1.20;
    }
}

public class VehicleRentalSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        for (int i = 0; i < n; i++) {
            int type = sc.nextInt();
            String vehicleNumber = sc.next();
            double rentPerDay = sc.nextDouble();
            int days = sc.nextInt();

            Vehicle vehicle;

            if (type == 1) {
                vehicle = new Car(vehicleNumber, rentPerDay);
            } else if (type == 2) {
                vehicle = new Bike(vehicleNumber, rentPerDay);
            } else {
                vehicle = new Truck(vehicleNumber, rentPerDay);
```

```
            }

            double totalRent = vehicle.calculateRent(days);

            System.out.printf("%s %.2f%n",
                    vehicle.vehicleNumber, totalRent);
        }

        sc.close();
    }
}
```

SAMPLE INPUT:

```
3
1 CAR101 1000 5
2 BIKE201 500 4
3 TRUCK301 2000 3
```

Output

```
Plain text
CAR101 5000.00
BIKE201 1800.00
TRUCK301 7200.00
```

8.PROBLEM STATEMENT:

Design and implement a **Payment Processing System** using a Java **interface and abstract class**.

The system should support three payment methods:

- **Credit Card**
- **UPI**
- **Net Banking**

Create a Payment interface with a pay() method. Each payment method should implement this interface.

Create an abstract PaymentProcessor class with a processPayment() method. The OnlinePaymentProcessor class should extend it and calculate the final payment amount by adding the appropriate processing fee:

- **Credit Card:** 2%
- **UPI:** 1%
- **Net Banking:** 1.5%

The design should allow new payment methods to be added without modifying the existing PaymentProcessor class.

For each transaction, display the payment type and final amount rounded to **two decimal places**.

SOURCE CODE:

```
package com;

import java.util.Scanner;


interface Payment {
   void pay(double amount);
}
```

```
class CreditCardPayment implements Payment {

    public void pay(double amount) {
        System.out.println("Credit Card Payment: " + amount);
    }
}

class UPIPayment implements Payment {

    public void pay(double amount) {
        System.out.println("UPI Payment: " + amount);
    }
}

class NetBankingPayment implements Payment {

    public void pay(double amount) {
        System.out.println("Net Banking Payment: " + amount);
    }
}

abstract class PaymentProcessor {
    abstract double processPayment(Payment payment, double amount);
}

class OnlinePaymentProcessor extends PaymentProcessor {
```

```
    double processPayment(Payment payment, double amount) {

        double fee = 0;

        if (payment instanceof CreditCardPayment) {
            fee = amount * 0.02;
        }
        else if (payment instanceof UPIPayment) {
            fee = amount * 0.01;
        }
        else if (payment instanceof NetBankingPayment) {
            fee = amount * 0.015;
        }

        payment.pay(amount);

        return amount + fee;
    }
}

public class PaymentProcessingSystem {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
```

```
int n = sc.nextInt();

PaymentProcessor processor = new OnlinePaymentProcessor();

for (int i = 0; i < n; i++) {

    int type = sc.nextInt();
    double amount = sc.nextDouble();

    Payment payment;
    String paymentType;

    if (type == 1) {
        payment = new CreditCardPayment();
        paymentType = "CreditCard";
    }
    else if (type == 2) {
        payment = new UPIPayment();
        paymentType = "UPI";
    }
    else {
        payment = new NetBankingPayment();
        paymentType = "NetBanking";
    }
```

```
            double finalAmount =
                    processor.processPayment(payment, amount);

            System.out.printf("%s %.2f%n",
                    paymentType, finalAmount);
        }

        sc.close();
    }
}
```

OUTPUT:

```
3
1 1000
Credit Card Payment: 1000.0
CreditCard 1020.00
2 2000
UPI Payment: 2000.0
UPI 2020.00
3 5000
Net Banking Payment: 5000.0
NetBanking 5075.00
```

9.PROBLEM STATEMENT:

Exception handling is the process of responding to the occurrence, during computation, of exceptions – anomalous or exceptional conditions requiring special processing – often changing the normal flow of program execution.

Using the try statement, we can test a block of code for errors. The catch block contains the code that specifies what to do if an exception occurs.

You will be given two 32-bit signed integers, x and y, as input. You have to compute x/y.

- If x and y are not 32-bit signed integers, or if y is zero, an exception will occur and you must report it.

SOURCE CODE:

```
import java.io.*;
import java.util.*;

public class Solution {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        try {
            int x = scanner.nextInt();
            int y = scanner.nextInt();
            System.out.println(x / y);
        } catch (InputMismatchException e) {
            System.out.println("java.util.InputMismatchException");
        } catch (ArithmeticException e) {
            System.out.println(e);
        } finally {
            scanner.close();
        }
    }
}
```

OUTPUT:

Input (stdin)

```
10
3
```

Your Output (stdout)

```
3
```

Expected Output

```
3
```